

🚧 RS003 Week 9 — English Worksheet

Topic: Keep the Ship Inside the Screen · **Course:** Pygame Space Shooter · **Time:** about 45 minutes

This week you think about how to stop the ship from flying off the screen on any side. You set a **minimum** and **maximum** for both X and Y, then use comparisons (`<` , `>`) to keep the ship inside all four walls. The trick of forcing a value back inside a range is called **clamping**. In this worksheet you read code, fix code, and explain code.

💬 Words to know: **minimum** / **maximum**, **compare** (`<` , `>`), **clamp**, **boundary**, **limit**

1 · Predict 🧠

Read each piece of code. Write what you think it does. Just think it through in your head — no running.

```
if ship_x < 20: ship_x = 20
if ship_x > WIDTH - 20: ship_x = WIDTH - 20
```

These two lines run **AFTER** the ship moves. What do they do if the ship went too far left or too far right?


```
if ship_y < 20: ship_y = 20
if ship_y > HEIGHT - 60: ship_y = HEIGHT - 60
```

These keep the ship inside the top and bottom. Why is the bottom limit `HEIGHT - 60` and not just `HEIGHT` ?


```
pygame.draw.rect(screen, RED, (0, 0, WIDTH, HEIGHT), 3)
```

A red rectangle outline is drawn around the whole edge of the screen. What does it show the player?


```
keys = pygame.key.get_pressed()
if keys[pygame.K_LEFT]: ship_x -= SPEED
if keys[pygame.K_RIGHT]: ship_x += SPEED
```

If the player holds the LEFT arrow, what happens to `ship_x` each frame? Would the ship leave the screen without the clamp lines?

2 · Spot the Bug 🐛

Each block below was meant to do something but is broken. Write the fixed code, then explain why the original was wrong.

Bug A — The ship should not fly past the left wall, but it flies right off the left edge of the screen.

```
if keys[pygame.K_LEFT]:  
    ship_x -= SPEED
```

Hint: there is no clamp pulling the ship back if it goes past the left wall.

Write the fixed code (add the left clamp):

Why was it wrong? Why does your fix work?

Bug B — The clamp is supposed to pull the ship back if it goes too far, but the comparisons are backwards so it teleports to the wrong wall.

```
if ship_x > 20:  
    ship_x = 20  
if ship_x < WIDTH - 20:  
    ship_x = WIDTH - 20
```

Hint: if the ship is **smaller** than 20 it went too far left. Check the directions.

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug C — The ship is clamped left and right, but it still flies off the **top** and **bottom** of the screen.

```
if ship_x < 20: ship_x = 20
if ship_x > WIDTH - 20: ship_x = WIDTH - 20
```

Hint: there are only two clamps here. You need two more for `ship_y`.

Write the fixed code (add the top and bottom clamps):

Why was it wrong? Why does your fix work?

3 · Explain the Code

Read this working example. It is the full ship-stays-on-screen program. Answer the questions about it.
Reading only — no running.

```
import pygame
pygame.init()

WIDTH, HEIGHT = 800, 600
CENTER_X = WIDTH // 2
screen = pygame.display.set_mode((WIDTH, HEIGHT))

ship_x = CENTER_X
ship_y = HEIGHT - 100
SPEED = 5
RED = (230, 60, 60)

clock = pygame.time.Clock()
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]: ship_x -= SPEED
    if keys[pygame.K_RIGHT]: ship_x += SPEED
    if keys[pygame.K_UP]: ship_y -= SPEED
    if keys[pygame.K_DOWN]: ship_y += SPEED

    if ship_x < 20: ship_x = 20
    if ship_x > WIDTH - 20: ship_x = WIDTH - 20
    if ship_y < 20: ship_y = 20
    if ship_y > HEIGHT - 60: ship_y = HEIGHT - 60

    screen.fill((10, 12, 40))
    pygame.draw.rect(screen, RED, (0, 0, WIDTH, HEIGHT), 3)
    pygame.draw.rect(screen, (80, 220, 255), (ship_x - 20, ship_y, 40, 50))
    pygame.draw.polygon(screen, (255, 255, 255), [(ship_x, ship_y - 20), (ship_x - 20, ship_y), (ship_x + 20, ship_y)])
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

What do the lines `ship_x = CENTER_X` and `ship_y = HEIGHT - 100` decide?

The four `if keys[...]` lines come BEFORE the four clamp lines. Why is that order important?

Point to the four clamp lines. Which one stops the ship at the right wall, and what is its maximum?

What would change on the screen if you deleted the line `pygame.draw.rect(screen, RED, (0, 0, WIDTH, HEIGHT), 3)` ?

`SPEED` is set to 5. If you made it 20, would the clamps still keep the ship inside? Why or why not?

4 · Explain Your Lesson Code 🎥

In today's live lesson you wrote your own ship-stays-on-screen code. Now record a short video on your phone where you explain the code you wrote. You can show it running on screen if you like. Try to use these words: **minimum**, **maximum**, **compare**, **clamp**, **boundary**.

1. Show your code and point to your four clamp lines.
2. Read one clamp line out loud and explain what it does.
3. Explain where the ship stops and why.
4. Say what would happen with no limits at all.

Write what you will say in your video. Plan it here before you record.

Submit 

Send this worksheet + a video explaining your lesson code to teacher on KakaoTalk.